



2-Month Frontend + Python AI/ML Roadmap & Notes

HTML - CSS - JavaScript - Advanced JS - Python - Django - ML - AI - LLM - n8n - Make.com

For boys and girls who want to start confidently and become job-ready interns.
Tagline: Where Creativity Meets Technology

"Learn the basics deeply, build projects weekly, and become useful in real-world teams."

Training Area	Outcome
Frontend Foundation	Build clean, responsive interfaces with HTML, CSS, JavaScript, DOM, APIs, and advanced JS concepts.
Python Foundation	Write clean Python scripts, work with files, OOP, exceptions, modules, virtual environments, and automation.
Django Skills	Build web apps, admin panels, models, views, templates, authentication, and REST APIs with Django/DRF.
AI/ML Skills	Understand data preprocessing, ML models, evaluation, NLP basics, embeddings, LLM concepts, and prompt engineering.
Automation Skills	Use n8n and Make.com for workflow automation, API integrations, webhooks, and AI-powered automations.
Final Delivery	Complete portfolio-ready projects and one final AI web application.

What Changed From the MERN Roadmap

This version keeps the frontend foundation from the previous roadmap: HTML, CSS, JavaScript, DOM, and Advanced JavaScript. It removes React JS, Node.js, Express, and MongoDB, then replaces that backend path with Python, Django, Machine Learning, AI, LLM, n8n, and Make.com training.

Training Goal

- First build strong frontend basics so students can create clean web pages and interactive UI.
- Then move into Python for scripting, data handling, automation, and backend concepts.
- Use Django to build practical web applications and APIs.
- Introduce Machine Learning and AI concepts through implementation-first practice.
- Finish with LLM apps, prompt engineering, workflow automation, deployment, and portfolio projects.

Daily Study Formula

- 60 to 90 minutes: concept learning and notes.
- 90 to 120 minutes: hands-on coding practice.
- 30 minutes: revision, GitHub push, and small summary of what was learned.
- Weekly target: one mini-project and one presentation/demo.
- Rule: no topic is complete until a student builds a small working example.

Motivation: "Confidence comes from practice, not from watching tutorials only."

2-Month Roadmap Overview

This is an 8-week training program with a practical focus. The roadmap can be used for a classroom batch, internship batch, or self-paced training. Students should commit 3 to 4 hours daily.

Days	Main Focus	Key Outcomes	Mini Project / Practice
1-5	HTML foundation	Document structure, semantic HTML, forms, tables, media, SEO and accessibility basics.	Personal profile page + contact form
6-12	CSS foundation	Selectors, box model, typography, flexbox, grid, responsive design, transitions, animations, CSS architecture.	Responsive landing page
13-20	JavaScript beginner + DOM	Variables, types, operators, functions, arrays, objects, events, DOM, forms, localStorage.	Calculator, quiz app, or task list
21-27	Advanced JavaScript	Scope, hoisting, closures, this, promises, async/await, fetch, modules, event loop, debounce/throttle.	Weather app/API dashboard
28-34	Python foundation	Syntax, data types, collections, loops, functions, files, exceptions, modules, virtual environments.	Python automation scripts
35-41	Advanced Python + Django basics	OOP, decorators, generators, async basics, Django project, apps, URLs, views, templates, models.	Django blog/student portal
42-48	Django REST + ML foundation	Django forms/auth/admin, DRF APIs, pandas/numpy, preprocessing, supervised/unsupervised learning.	Prediction API or analytics dashboard
49-55	ML, AI and LLM apps	Regression, classification, evaluation, NLP, embeddings, vector DB, prompt engineering, LLM APIs.	AI smart assistant prototype
56-60	n8n, Make.com, deployment and final project	Webhooks, workflow automation, AI automations, deployment, documentation, presentation.	Final AI web app + automation workflow

Detailed 60-Day Training Plan

Week 1 - HTML Foundation

Day	Tasks
Day 1	Web basics, browser, client/server, VS Code setup, folder structure, first HTML page.
Day 2	HTML document structure: html, head, body, title, meta, headings, paragraphs, links.
Day 3	Lists, tables, images, audio, video, iframe, link targets: _blank, _self, _parent, _top.
Day 4	Forms: input types, labels, textarea, select, fieldset, required, pattern, min, max.
Day 5	Semantic HTML, accessibility basics, alt text, labels, aria basics, data attributes, SEO meta tags.

Week 2 - CSS Foundation

Day	Tasks
Day 6	CSS intro, inline/internal/external CSS, selectors, specificity, colors, units: px, %, em, rem.
Day 7	Typography, font, line-height, box model: margin, padding, border, content.
Day 8	Display: block, inline, inline-block; position: static, relative, absolute, fixed, sticky.
Day 9	Flexbox: direction, wrap, justify-content, align-items, gap; build navbar and cards.
Day 10	CSS Grid: columns, rows, gap, grid areas; compare flexbox vs grid.
Day 11	Responsive design, media queries, breakpoints, mobile-first layout.
Day 12	Transitions, hover effects, transforms, keyframes, CSS variables, BEM architecture.

Week 3 - JavaScript Beginner + DOM

Day	Tasks
Day 13	What is JavaScript, comments, console, variables: var, let, const.
Day 14	Data types, type conversion, operators, comparison, logical, assignment, equality.
Day 15	Conditionals: if, else, switch; loops: for, while, do-while.
Day 16	Functions, parameters, arguments, return statement, scope basics.
Day 17	Arrays, array methods, objects, properties, methods, nested objects.
Day 18	String methods, Math object, Date object, strict mode.
Day 19	DOM tree, selecting elements, manipulating styles/content/attributes, creating elements.
Day 20	Events, form handling, event bubbling/capturing/delegation, input events, DOMContentLoaded.

Week 4 - Advanced JavaScript

Day	Tasks
Day 21	Execution context, call stack, lexical scope, hoisting.
Day 22	Closures, this keyword, arrow functions, default parameters.
Day 23	Rest/spread, destructuring, higher-order functions, callbacks.
Day 24	Promises, async/await, try/catch/finally, API error handling.
Day 25	Event loop, microtasks/macrotasks, timers, localStorage, sessionStorage, cookies.
Day 26	JSON, fetch API, CORS basics, modules import/export, shallow vs deep copy.
Day 27	Debounce, throttle, prototypes, prototype chain, call/apply/bind, currying.

Week 5 - Python Foundation

Day	Tasks
Day 28	Python setup, VS Code, pip, virtual environments, variables, syntax, input/output.
Day 29	Data types: strings, numbers, booleans, lists, tuples, sets, dictionaries.
Day 30	Conditionals, loops, list comprehensions, useful built-in functions.

Day	Tasks
Day 31	Functions, parameters, return values, recursion, lambda basics.
Day 32	File handling with TXT, CSV, JSON; reading/writing data safely.
Day 33	Exception handling, logging basics, modules, packages, requirements.txt.
Day 34	Mini project: data cleaner or smart automation script.

Week 6 - Advanced Python + Django Basics

Day	Tasks
Day 35	OOP: classes, objects, constructors, methods, inheritance, encapsulation.
Day 36	Decorators, generators, iterators, context managers, type hints.
Day 37	Async basics, multithreading/multiprocessing concepts, when to use each.
Day 38	Django setup, project/app structure, settings, URLs, views, templates.
Day 39	Django models, migrations, admin panel, ORM basics.
Day 40	Django forms, validation, messages, static files, media files.
Day 41	Django authentication basics, login/logout, permissions; mini Django CRUD project.

Week 7 - Django REST + ML Foundation

Day	Tasks
Day 42	Django REST Framework setup, serializers, views/viewsets, routers.
Day 43	API CRUD, permissions, authentication tokens/JWT concept, API testing.
Day 44	NumPy and pandas basics, dataframes, cleaning missing values, CSV/Excel handling.
Day 45	ML overview: supervised vs unsupervised learning, train/test split, feature engineering.
Day 46	Regression models: linear regression, logistic regression; evaluation metrics.
Day 47	Classification models: KNN, decision tree, random forest, model comparison.
Day 48	Save/load models, create prediction API with Django/DRF or FastAPI concept.

Week 8 - AI, LLM, n8n, Make.com and Deployment

Day	Tasks
Day 49	NLP basics, text preprocessing, tokens, embeddings, similarity search.
Day 50	Deep learning basics: neural networks, activation, loss, optimizers, CNN/RNN concepts.
Day 51	Transformers and LLM concepts, prompt engineering, system/user prompts, output formatting.
Day 52	Build LLM assistant prototype: prompt, API call, response handling, error handling.
Day 53	Vector database concept, RAG basics: documents, chunks, embeddings, retrieval, answer generation.
Day 54	n8n basics: nodes, triggers, webhooks, HTTP request, data mapping, AI workflow.
Day 55	Make.com basics: scenarios, modules, routers, filters, scheduling, webhook automation.

Final Polish Days

Day	Tasks
Day 56	Choose final project, define features, database/model/API plan, UI screens, workflow automation plan.
Day 57	Build final project core: Django app/API, frontend UI, ML or LLM feature.
Day 58	Add automation using n8n or Make.com: form leads, email alerts, AI summary, CRM/task update.
Day 59	Deployment, environment variables, security basics, README, screenshots, demo video.
Day 60	Final presentation, code review, portfolio update, improvement plan.

HTML Notes

HTML is the structure of a web page. Strong students must write semantic, accessible, and SEO-friendly HTML before focusing on design or JavaScript.

Topic	Must Know	Practice
Document Structure	html, head, body, title, meta tags, viewport, linking CSS/JS.	Create a clean index.html file from scratch.
Common Tags	Headings, paragraphs, links, images, div, span, lists, tables.	Build a profile page with sections and navigation.
Forms	input, label, textarea, select, fieldset, button, required, pattern, min/max.	Create registration and contact forms.
Semantic HTML	header, nav, main, section, article, aside, footer.	Convert div-based layout into semantic layout.
Accessibility	alt text, labels, keyboard-friendly forms, basic aria attributes.	Test page with keyboard and labels.
SEO Basics	title, meta description, heading hierarchy, canonical concept.	Write title/description for landing page.
Data Attributes	data-* attributes store custom values in HTML.	Use data-id for cards and access it in JS.

HTML Example

```
<header><nav><a href="#home">Home</a></nav></header>
<main><section><h1>Welcome</h1><p>My first semantic page.</p></section></main>
<footer>Copyright IIFA TECH</footer>
```

CSS Notes

CSS controls the visual design and layout. Learn selectors, spacing, typography, box model, responsive design, flexbox, and grid before moving into complex UI.

Topic	Explanation	Practice Task
Selectors	Element, class, id, descendant, child, pseudo-class, pseudo-element.	Style cards using class selectors and hover states.
Units	px, %, em, rem, vw, vh. Use rem for scalable fonts.	Create responsive typography with rem.
Box Model	Content, padding, border, margin.	Make consistent card spacing.
Display	block, inline, inline-block, none.	Build nav menu using inline-block/flex.
Position	static, relative, absolute, fixed, sticky.	Create sticky navbar and absolute badge.
Flexbox	Best for one-dimensional layout: row or column.	Build header, card row, centered login box.
Grid	Best for two-dimensional page layout.	Build product gallery grid.
Responsive	Media queries, breakpoints, mobile-first design.	Make landing page work on all screens.
Effects	transition, transform, animation, keyframes.	Add button hover and loader animation.
Architecture	BEM naming and organized CSS files.	Use card__title, card__image, card--featured.

CSS Tip

Use Flexbox when content flows in one direction. Use Grid when you need rows and columns together. Start mobile-first, then add breakpoints.

```
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 1rem; }
```

Motivation: "Clean design is not decoration; it is clear communication."

JavaScript Beginner Notes

JavaScript adds logic and interactivity. Students must understand the core language before using it with Python APIs or AI dashboards.

Topic	Must Know	Example Practice
Variables	var, let, const; prefer const unless reassignment is needed.	Store user name and show greeting.
Data Types	string, number, boolean, null, undefined, object, array.	Check type using typeof.
Operators	Arithmetic, comparison, logical, assignment, equality.	Create marks grading logic.
Conditionals	if, else, switch.	Show discount based on role.
Loops	for, while, do-while.	Print product list and totals.
Functions	parameters, arguments, return, arrow functions.	Create tax calculation function.
Scope	Global, local, block scope.	Compare let and var behavior.
Arrays	push, pop, map, filter, reduce, find, includes.	Filter completed tasks.
Objects	Properties, methods, nested objects.	Create user profile object.
Strings/Math/Date	String methods, Math object, Date object.	Format date and random OTP.

DOM Notes

- DOM means Document Object Model. It represents HTML as a tree that JavaScript can read and update.
- Important methods: `querySelector`, `querySelectorAll`, `getElementById`, `createElement`, `appendChild`, `classList`, `setAttribute`.
- Events make pages interactive: `click`, `input`, `submit`, `change`, `keyup`, `DOMContentLoaded`.
- Event delegation is useful when many child items share the same handler, such as task list buttons.
- Form handling includes reading inputs, validating values, preventing default submit, and showing messages.

JS Example

```
const btn = document.querySelector("#saveBtn");
btn.addEventListener("click", () => {
  const name = document.querySelector("#name").value;
  document.querySelector("#result").textContent = `Hello ${name}`;
});
```


Advanced JavaScript Notes

Advanced JavaScript strengthens problem-solving and prepares students for API work, dashboards, AI tools, and workflow automation.

Topic	Simple Meaning	Where It Helps
Execution Context	Environment where JS code runs.	Understanding how functions execute.
Call Stack	Tracks function calls.	Debugging function flow and errors.
Lexical Scope	Inner functions can access outer variables.	Closures and reusable functions.
Hoisting	Declarations are conceptually available before execution.	Avoiding var/function confusion.
Closures	Function remembers outer variables after outer function finishes.	Private variables, debounce, factories.
this Keyword	Value depends on how function is called.	Object methods, event handlers.
Rest/Spread	Collect values or copy/expand arrays/objects.	Clean updates and function args.
Destructuring	Extract values from arrays/objects.	Cleaner API data handling.
Promises	Represent future async result.	API calls.
Async/Await	Cleaner way to handle promises.	Fetching backend or AI data.
Event Loop	Manages async tasks in single-threaded JS.	Timers, promises, UI responsiveness.
Fetch & CORS	Fetch gets data; CORS controls cross-origin requests.	Frontend/backend integration.
Debounce/Throttle	Control how often a function runs.	Search input, scroll events.
Prototype Chain	Object inheritance system in JS.	Understanding JS internals.

Motivation: "Advanced JS becomes simple when every topic has a small example."

Python Notes - Beginner to Advanced

Python is the main language for automation, backend development, data science, AI, and ML. Students should write clean scripts before jumping into libraries.

Topic	What to Learn	Practice
Setup	Python, pip, venv, VS Code, requirements.txt.	Create virtual environment and install packages.
Syntax	Indentation, comments, variables, input/output.	Build a CLI greeting app.
Data Types	str, int, float, bool, list, tuple, set, dict.	Store student data and print summary.
Control Flow	if/else, for, while, break, continue.	Marks grading system.
Functions	Parameters, return, recursion, lambda.	Reusable calculator functions.
File Handling	Read/write TXT, CSV, JSON, Excel concept.	Clean CSV and export report.
Exceptions	try/except/finally, custom error messages.	Handle invalid file and input errors.
OOP	Class, object, constructor, methods, inheritance.	Student/Employee management classes.
Advanced	Decorators, generators, iterators, context managers.	Logging decorator and data generator.
Async/Threads	Async/await, threading/multiprocessing concepts.	Run slow tasks and compare behavior.

Python Mini Example

```
import csv

def read_students(path):
    with open(path, newline="") as file:
        return list(csv.DictReader(file))

students = read_students("students.csv")
print(f"Total students: {len(students)}")
```

Django Notes

Django is a Python web framework used to build secure and scalable web applications quickly. It includes routing, templates, ORM, admin panel, forms, authentication, and security features.

Topic	Meaning	Practice
Project Setup	django-admin startproject, manage.py, settings.py.	Create first Django project.
Apps	Feature modules inside Django project.	Create blog or students app.
URLs	Route browser paths to views.	/students, /students/1.
Views	Functions/classes that handle requests and responses.	Return HTML page or JSON.
Templates	HTML files with Django template language.	List students in template.
Models	Python classes mapped to database tables.	Student, Course, Enrollment model.
Migrations	Apply model changes to database.	makemigrations and migrate.
Admin Panel	Built-in dashboard for managing data.	Register models in admin.py.
Forms	Validate and save user input.	Create student form.
Authentication	Login, logout, permissions.	Protected dashboard.
Django REST Framework	Build APIs with serializers, viewsets, routers.	Students CRUD API.
Deployment Basics	Static files, environment variables, debug false.	Deploy simple Django app.

Django Commands

```
python -m venv venv
source venv/bin/activate # Mac/Linux
pip install django djangorestframework
django-admin startproject config .
python manage.py startapp students
python manage.py runserver
```

Django REST API Notes

Django REST Framework helps build APIs that frontend pages, mobile apps, or AI tools can call. Students should understand serializers, request/response, authentication, and API testing.

API Part	Purpose	Example
Serializer	Converts model data to JSON and validates input.	StudentSerializer.
APIView/ViewSet	Handles HTTP methods and CRUD logic.	StudentViewSet.
Router	Auto-generates API routes for viewsets.	/api/students/.
Permissions	Controls who can access API.	IsAuthenticated.
Authentication	Identifies logged-in user.	Session/JWT/token auth concept.
Pagination	Splits large data into pages.	?page=2.
Filtering/Search	Find data by query.	?name=Ali&course;=AI.
Postman Testing	Test API without frontend.	GET, POST, PUT, DELETE requests.

Professional API Response Pattern

```
{
  "success": true,
  "message": "Student created successfully",
  "data": { "id": 1, "name": "Ayesha" }
}
```

Machine Learning Notes

Machine Learning teaches computers to find patterns in data and make predictions. Students should focus on business use cases and practical model building before deep theory.

Topic	Meaning	Practice
ML Overview	Learning patterns from data.	Predict sales, prices, marks, churn.
Supervised Learning	Training data has labels/answers.	Regression and classification.
Unsupervised Learning	No labels; find groups/patterns.	Customer segmentation.
Data Preprocessing	Cleaning data before training.	Handle missing values and duplicates.
Feature Engineering	Creating useful inputs for model.	Extract month from date.
Train/Test Split	Separate data for training and evaluation.	80/20 split.
Regression	Predict numeric value.	Price or sales prediction.
Classification	Predict category/class.	Spam/not spam, pass/fail.
Models	KNN, decision tree, random forest, linear/logistic regression.	Compare accuracy and errors.
Evaluation	Accuracy, precision, recall, F1, MAE, RMSE.	Choose metric by problem type.
Cross Validation	Test model stability on multiple splits.	Improve confidence in model.
Imbalanced Data	One class much larger than another.	Use class weights or resampling concept.
Model Saving	Save trained model for later use.	joblib or pickle.

ML Workflow

- Define the business problem clearly.
- Collect or load dataset from CSV, Excel, API, or database.
- Clean data and select useful features.
- Train multiple models and compare results.
- Save the best model and expose it through an API.
- Monitor predictions and improve with better data.

AI and Deep Learning Notes

AI is a broad field for building systems that perform tasks that usually need human intelligence. Deep learning uses neural networks and is especially useful for images, text, speech, and complex patterns.

Topic	Simple Meaning	Practice/Example
AI	Systems that perform intelligent tasks.	Chatbot, recommendation, OCR, assistant.
Neural Networks	Layers of connected units that learn patterns.	Simple classifier concept.
Activation Functions	Add non-linearity to model.	ReLU, sigmoid concept.
Loss Functions	Measure how wrong the model is.	Lower loss means better fit.
Optimizers	Update model weights to improve results.	Adam/SGD concept.
Regularization	Reduce overfitting.	Dropout, early stopping concept.
CNN	Used mainly for image tasks.	Image classification concept.
RNN/LSTM	Used for sequences/time series.	Text/time series concept.
NLP	AI for text language tasks.	Sentiment analysis, summarization.
Transformers	Architecture behind modern LLMs.	Attention-based text understanding.

Motivation: "AI is not magic. It is data, patterns, logic, experiments, and consistent improvement."

LLM and Prompt Engineering Notes

LLM means Large Language Model. LLMs can understand and generate text, summarize, answer questions, classify, translate, reason over documents, and power AI assistants.

Topic	Must Know	Practice
LLM Basics	Model predicts useful text based on prompt and context.	Ask model to summarize a paragraph.
Prompt Engineering	Write clear instructions, role, context, constraints, and output format.	Create prompt templates.
System/User Prompt	System sets behavior; user asks task.	Create assistant with rules.
Few-Shot Prompting	Give examples before asking output.	Classify support tickets.
Structured Output	Ask for JSON/table/checklist output.	Generate clean JSON for app.
Temperature	Controls creativity vs consistency.	Compare low vs high temperature outputs.
Embeddings	Convert text into numerical vectors.	Search similar documents.
Vector Database	Stores embeddings for similarity search.	FAQ/document search concept.
RAG	Retrieval Augmented Generation: retrieve documents then answer.	Company policy Q&A; assistant.
Safety/Evaluation	Check hallucination, privacy, correctness, and bias.	Review answer with source/context.

Good Prompt Template

Role: You are a helpful training assistant.
Task: Explain the topic to beginners.
Context: Students know basic Python only.
Rules: Use simple language, examples, and short steps.
Output: Give definition, example, practice task, and common mistakes.

RAG and AI Assistant Notes

RAG allows an AI assistant to answer using your own documents. It reduces guessing by retrieving relevant chunks before generating the response.

Step	What Happens	Tools/Concepts
1. Collect Documents	PDFs, docs, website pages, FAQs, CSV files.	Document loaders.
2. Chunk Text	Split content into smaller sections.	Chunks with overlap.
3. Create Embeddings	Convert chunks into vectors.	Embedding model.
4. Store Vectors	Save vectors for similarity search.	Vector DB concept.
5. Retrieve Context	Find the most relevant chunks for the question.	Top-k search.
6. Generate Answer	LLM answers using retrieved context.	Prompt + context.
7. Evaluate	Check correctness, missing context, hallucination.	Manual review and test questions.

AI Assistant Project Features

- Upload or paste training notes/documents.
- Ask questions from the document content.
- Return answer with short summary and source/context notes.
- Save chat history for review.
- Add admin settings for prompt instructions.
- Use Django or FastAPI as backend service.

n8n Learning Notes

n8n is a workflow automation tool. It helps connect apps, APIs, webhooks, databases, and AI services without writing full custom backend code for every automation.

Topic	Meaning	Practice
Workflow	A sequence of connected automation steps.	Create lead notification workflow.
Node	One action or step in a workflow.	HTTP request, email, AI node.
Trigger	Starts a workflow.	Webhook, schedule, form submit.
Webhook	URL that receives data from external app.	Receive website contact form data.
HTTP Request	Call external API from workflow.	Send data to Django API.
Data Mapping	Move data fields between nodes.	Map name/email/message.
IF/Switch	Conditional logic.	If lead source is website, send email.
AI Node/LLM	Use AI to summarize/classify/generate text.	Summarize form message.
Error Handling	Handle failed steps safely.	Send failure alert to admin.
Credentials	Secure API keys and access tokens.	Store email/API credentials safely.

n8n Practice Workflow

- Webhook receives contact form lead.
- AI summarizes the message and identifies priority.
- If priority is high, send email/Slack/WhatsApp alert.
- Save lead to Google Sheet or database API.
- Send auto-reply email to the user.

Make.com Learning Notes

Make.com is a visual automation platform used to create scenarios that connect different apps and services. It is useful for business workflows, CRM updates, notifications, reporting, and AI-powered operations.

Topic	Meaning	Practice
Scenario	Automation workflow in Make.com.	Create lead capture scenario.
Module	A single app/action step.	Google Sheets module, email module.
Trigger Module	Starts scenario.	Webhook or scheduled trigger.
Router	Splits flow into multiple paths.	Different path for support/sales leads.
Filter	Condition that controls whether next module runs.	Only send alert if priority is high.
Iterator/Aggregator	Handle lists and combine data.	Process multiple rows from sheet.
Webhook	Receive data from website/app/API.	Website form to Make scenario.
HTTP Module	Call custom API.	Send prediction request to Django API.
Scheduling	Run automation on time interval.	Daily report at 9 AM.
AI Integration	Use AI for summary/classification.	Generate daily sales summary.

Make.com Practice Scenario

- Trigger from webhook or Google Form.
- Validate required fields.
- Send data to Google Sheets and Django API.
- Use AI to summarize the lead or classify interest.
- Send confirmation email and team notification.
- Add error path for missing data or failed API call.

Practice Projects

Level	Project	Skills Covered	Required Features
HTML/CSS	Responsive Portfolio Website	Semantic HTML, CSS layout, responsive design.	Navbar, hero, about, skills, projects, contact form, mobile view.
HTML/CSS	Product Landing Page	Forms, cards, grid/flexbox, SEO basics.	Product sections, pricing cards, FAQ, CTA buttons.
JavaScript	Calculator / BMI App	Functions, events, DOM, validation.	Input validation, calculation result, reset button.
JavaScript	Quiz App	Arrays, objects, conditionals, DOM, score logic.	Questions, options, score, restart, result message.
Advanced JS	Weather/API Dashboard	Fetch API, async/await, loading/error states.	Search city/data, show results, handle invalid input.
Python	Smart Automation Script	Files, CSV/JSON, functions, exceptions, logging.	Import data, clean data, validate rows, export report.
Django	Student/Blog Management App	Models, views, templates, forms, auth, admin.	CRUD, login, dashboard, search/filter.
Django REST	Prediction API Backend	DRF, serializers, APIs, model loading.	Send input and return ML prediction.
ML	Sales/Price Prediction System	Pandas, preprocessing, regression/classification.	Train model, compare results, save model, build API.
LLM	AI Smart Assistant	Prompting, embeddings, RAG, API calls.	Ask questions from notes/docs and get structured answers.
Automation	n8n or Make Lead Workflow	Webhooks, API calls, AI summary, notifications.	Capture lead, classify, save, notify, auto-reply.
Final	AI Web Application	Frontend + Django + AI/ML + Automation.	Dashboard, API, AI feature, automation, deployment, README.

Final Project Ideas

Option 1 - AI Student Support Assistant

- Django dashboard for admin and students.
- Upload training notes and ask questions using LLM/RAG concept.
- Student progress tracking and feedback form.
- n8n or Make.com workflow sends weekly progress email to trainer.
- Frontend includes clean HTML/CSS/JS pages and responsive dashboard.

Option 2 - AI Sales Prediction and Lead Automation System

- Upload sales CSV and train/use prediction model.
- Django API returns prediction and business summary.
- LLM generates short explanation of prediction for non-technical users.
- Make.com or n8n captures website leads and sends priority alerts.
- Final dashboard shows leads, predictions, and reports.

Option 3 - Smart HR Resume Screening Tool

- Upload candidate resumes or form data.
- Use Python text processing and LLM prompt to summarize candidate skills.
- Django dashboard for HR/admin review.
- Automation sends shortlisted candidates to email/Google Sheet.
- Add filters by skills, experience, and role.

Option 4 - AI Customer Support Ticket Classifier

- Frontend form submits customer issue.
- Python/ML or LLM classifies issue type and priority.
- Django admin panel stores tickets and status.
- n8n/Make sends urgent ticket alerts to team.
- Dashboard includes search, filters, and status updates.

Interview and Assessment Checklist

Area	Student Should Explain	Student Should Build/Demo
HTML	Semantic tags, forms, accessibility, SEO meta basics.	Responsive page with form.
CSS	Box model, flexbox, grid, responsive design, position, animations.	Landing page or portfolio.
JavaScript	Variables, functions, arrays, objects, DOM, events, fetch.	Interactive app with API.
Advanced JS	Closures, promises, async/await, event loop, modules, debounce.	API dashboard with loading/error states.
Python	Data types, functions, files, exceptions, OOP, modules.	Automation script with clean report.
Django	URLs, views, templates, models, migrations, admin, auth.	CRUD app with login.
DRF/API	Serializers, viewsets, routes, permissions, Postman.	Working REST API.
ML	Preprocessing, model training, evaluation, saving/loading model.	Prediction notebook or API.
LLM	Prompting, embeddings, RAG basics, structured output.	Small assistant prototype.
n8n/Make	Triggers, webhooks, APIs, filters, routers, error handling.	One working automation workflow.

Assessment Rules

- Every student must explain their code in simple words.
- Every mini-project must have a README, screenshots, and setup steps.
- Every API must be tested with sample request and response.
- Every AI/ML project must include dataset explanation, model choice, and limitation notes.
- Every automation must include trigger, action steps, success path, and failure path.

Tools and Setup Checklist

Category	Tools
Editor	VS Code with Python, Django, Prettier, ESLint/basic formatting extensions.
Frontend	Chrome/Edge DevTools, responsive mode, Lighthouse basics.
Python	Python 3, pip, venv, Jupyter Notebook, pandas, numpy, scikit-learn.
Django	Django, Django REST Framework, SQLite/PostgreSQL concept, django-environ.
AI/ML	scikit-learn, TensorFlow/PyTorch concept, embeddings/vector DB concept, LLM API provider concept.
Automation	n8n, Make.com, webhooks, HTTP module/node, Google Sheets/email integrations.
API Testing	Postman or Insomnia.
Version Control	Git and GitHub: commits, branches, README.
Deployment	Render/Railway/Vercel/Netlify concept, Docker basics, environment variables.

Security and Professional Habits

- Never commit API keys, tokens, passwords, or .env files to GitHub.
- Use environment variables for secrets.
- Validate user input in frontend and backend.
- Use authentication and permissions for private pages/APIs.
- Handle errors politely and log useful debugging information.
- Write clear README files and organize project folders professionally.

Learning Notes Summary

This training path is designed to create practical developers who can build a frontend page, write JavaScript logic, automate tasks with Python, create Django apps/APIs, understand ML/AI basics, and connect modern LLM and automation tools into real business workflows.

Skill	Final Ability
HTML/CSS	Create responsive, semantic, accessible, professional web pages.
JavaScript	Add interaction, API calls, validation, and dynamic UI behavior.
Advanced JavaScript	Understand asynchronous behavior and write cleaner real-world code.
Python	Automate tasks, process data, and build backend/AI foundations.
Django	Build web applications, admin dashboards, and REST APIs.
Machine Learning	Train, evaluate, and serve simple prediction models.
AI/LLM	Build AI assistant features using prompting, embeddings, and RAG concepts.
n8n/Make.com	Automate business workflows with webhooks, APIs, AI, and notifications.

Final Motivation: "A strong career is built one project at a time. Do not wait to become perfect. Build, improve, repeat."